

# IDone — Technical Architecture & Cryptography

**TECHNICAL SPECIFICATION:** Cryptographic Protocols, RFC 8785 Canonicalization, Zero-Knowledge Storage & EVM Anchors

|                           |                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------|
| Document Type             | Technical Architecture Whitepaper & Cryptographic Engineering Specification       |
| Cryptographic Primitives  | Ed25519 (Asymmetric Signatures), AES-256-GCM (Authenticated Encryption), Argon2id |
| Deterministic Standard    | RFC 8785 JSON Canonicalization Scheme (JCS) for Python & TypeScript Parity        |
| Blockchain Anchors        | Solidity 0.8.20 (IdentityRegistry.sol & CredentialStatus.sol on EVM)              |
| On-Chain Privacy Standard | Zero PII On-Chain: Cryptographic keccak256 Hashes and Merkle Roots Exclusively    |

## 1. High-Level System Architecture & Component Interaction

IDone is designed with an uncompromising defense-in-depth architecture. The platform decouples identity presentation, cryptographic verification, blinded persistent storage, and public ledger anchoring across four isolated execution tiers.

| Architecture Tier           | Technology Runtime                        | Security Responsibility & Guarantees                                                                 |
|-----------------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------|
| Tier 1: Client Application  | Next.js 14, TypeScript 5, WebCrypto       | Generates asymmetric keypairs, executes client AES-256-GCM encryption, filters selective disclosure. |
| Tier 2: Backend API Engine  | FastAPI, Python 3.11, Cryptography HAZMAT | Asynchronous request pipeline, RFC 8785 canonicalization, W3C DID document resolver, JWT issuer.     |
| Tier 3: Blind Storage Layer | PostgreSQL 16 / SQLite Fallback           | Stores blinded ciphertext, random 96-bit IVs, and 128-bit auth tags. Holds zero plaintext secrets.   |
| Tier 4: Blockchain Anchors  | EVM (Sepolia / Arbitrum / Polygon)        | Immutable revocation bitmaps and DID hash anchors. Zero personal identifiers ever touch the ledger.  |

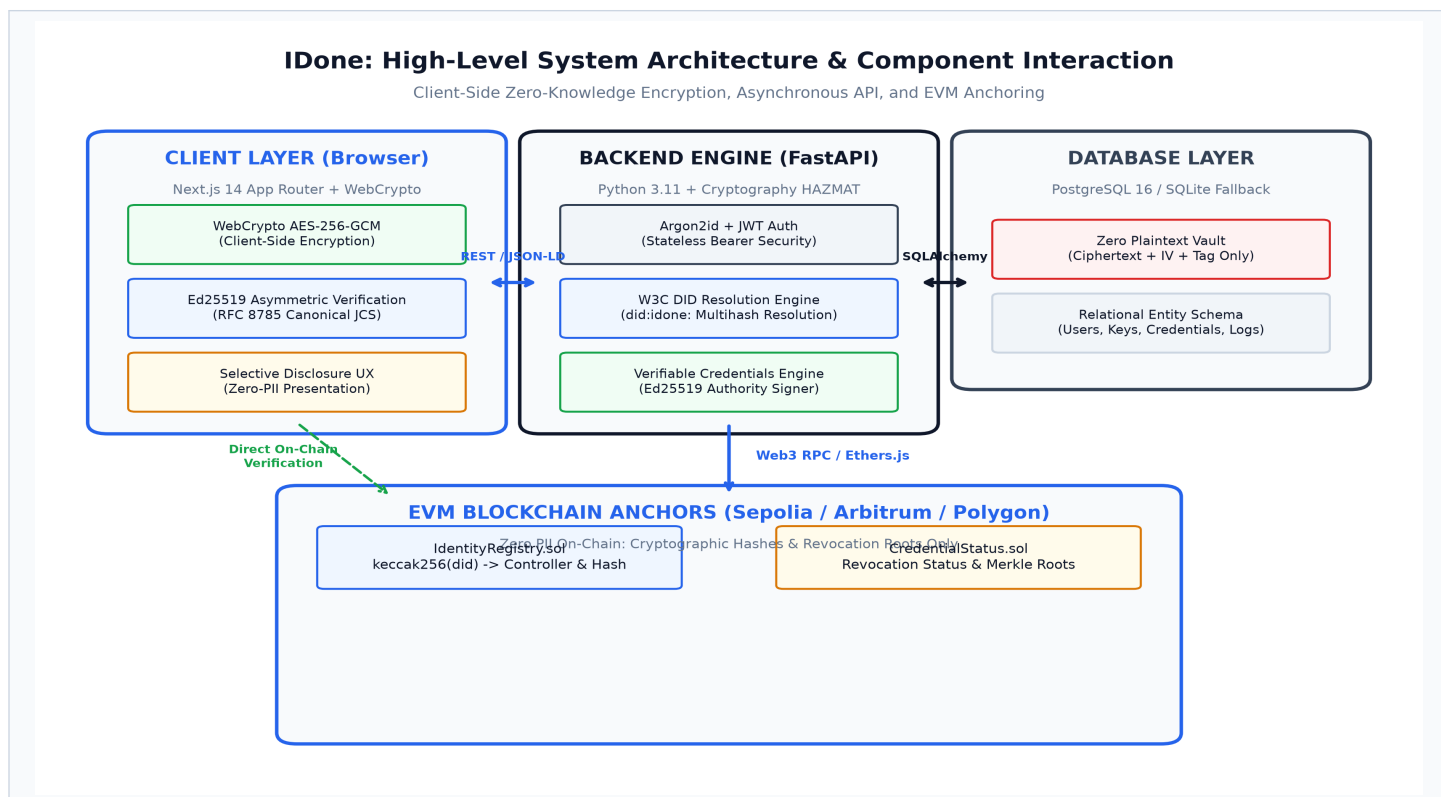


Figure 2.1: IDone High-Level System Architecture, Data Flow, and Trust Boundaries

## 2. Decentralized Identity (DID) Lifecycle & Ed25519 Key Rotation

Identity in IDone is defined by self-sovereign mathematical key ownership rather than centralized registry accounts. The identifier follows the format `did:idone:<sha256-hash>`, derived from the initial public key.

**Asymmetric Key Rotation Protocol:** In the event of device compromise or routine security key hygiene, the user triggers a key rotation. The backend identity engine generates a new Ed25519 keypair and creates a new assertion method (e.g. `#key-2`). Crucially, the historical key (`#key-1`) is retained in the resolved DID Document history as a deprecated method, ensuring that previously signed credentials remain mathematically verifiable while future credentials must be signed by the active key.

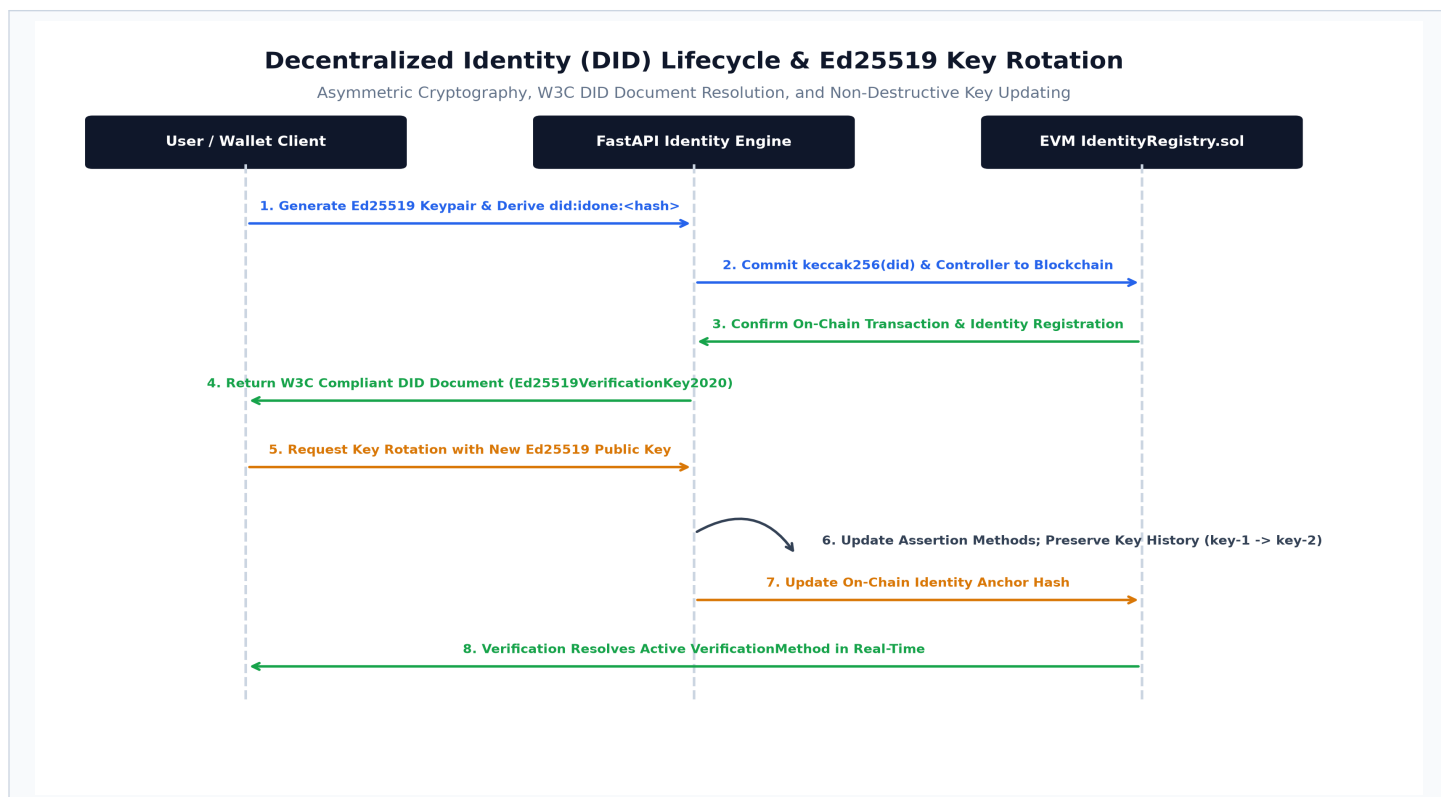


Figure 2.2: Sequence Diagram of Asymmetric Key Generation, Resolution, and Non-Destructive Key Rotation

### 3. W3C Verifiable Credentials & RFC 8785 Canonicalization (JCS)

A foundational challenge in decentralized identity is **cross-language signature verification**. In standard JSON, key ordering, whitespace, and floating-point representations vary between Python and JavaScript engines. If Python signs a JSON string, a TypeScript verifier re-serializing that JSON might produce a differing byte stream, causing signature verification to fail.

**DETERMINISTIC CRYPTOGRAPHIC GUARANTEE: THE RFC 8785 SOLUTION:** IDone implements the **JSON Canonicalization Scheme (RFC 8785)** across both Python and TypeScript. All whitespace is stripped, object keys are sorted lexicographically by Unicode code point, and characters are strictly UTF-8 encoded. This guarantees an identical SHA-256 digest regardless of execution platform.

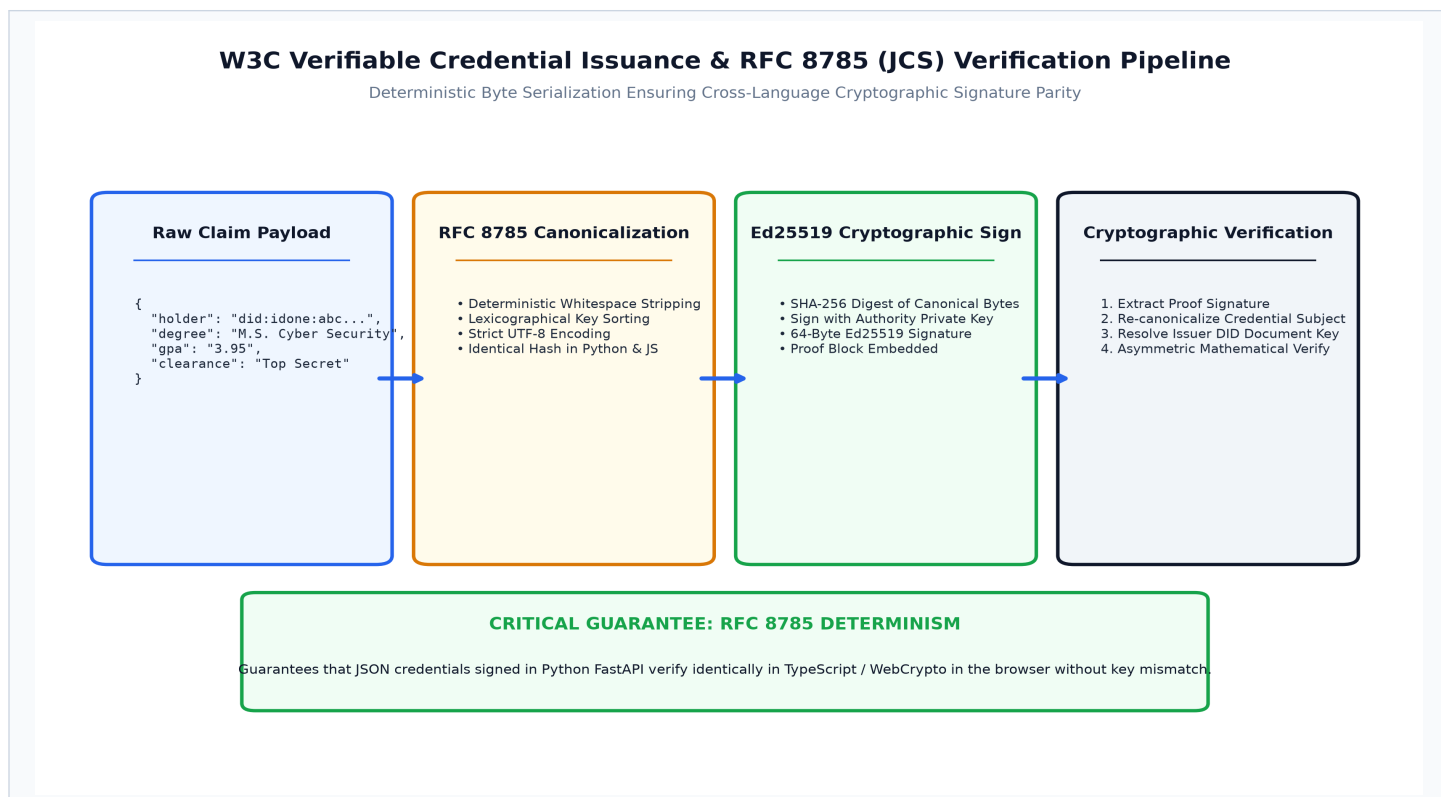
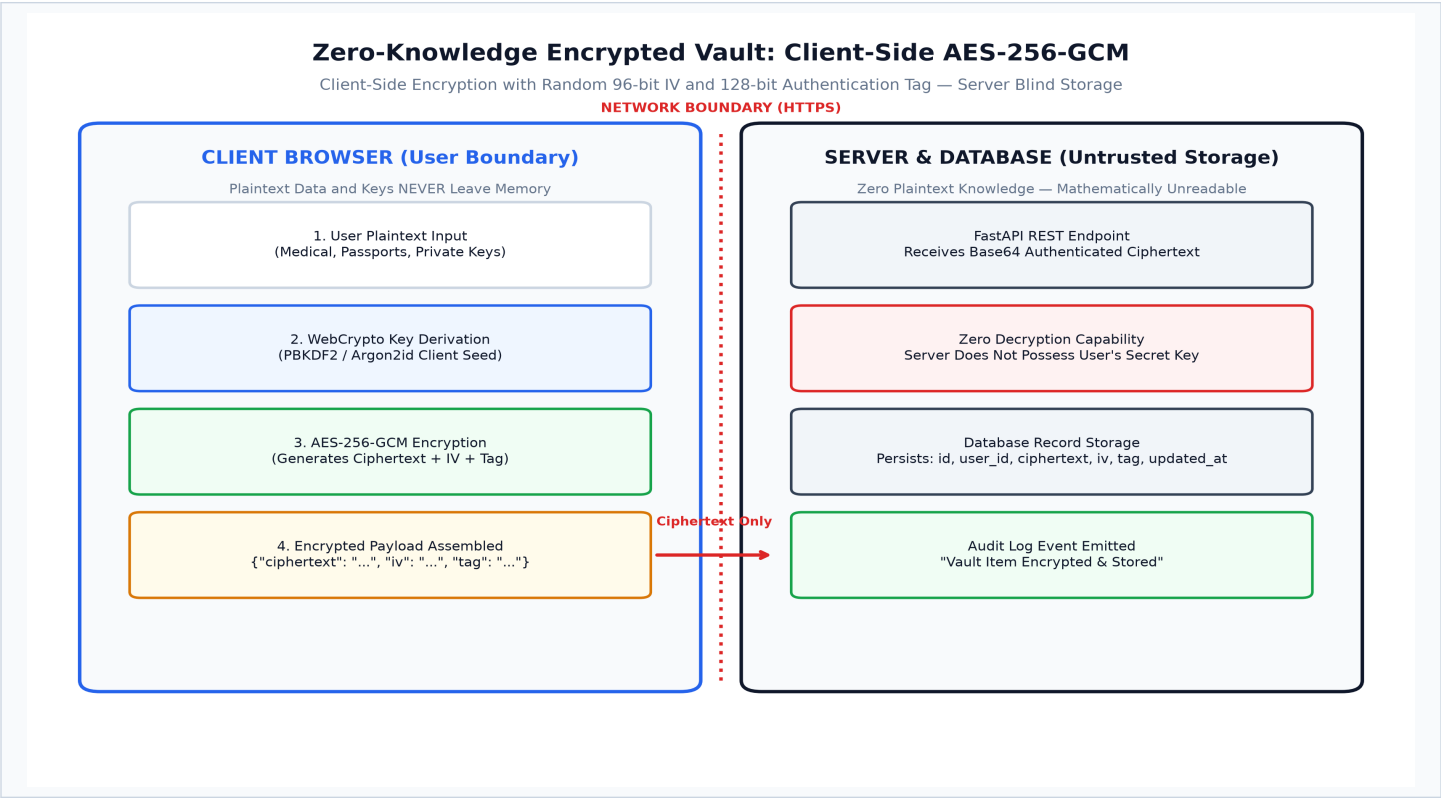


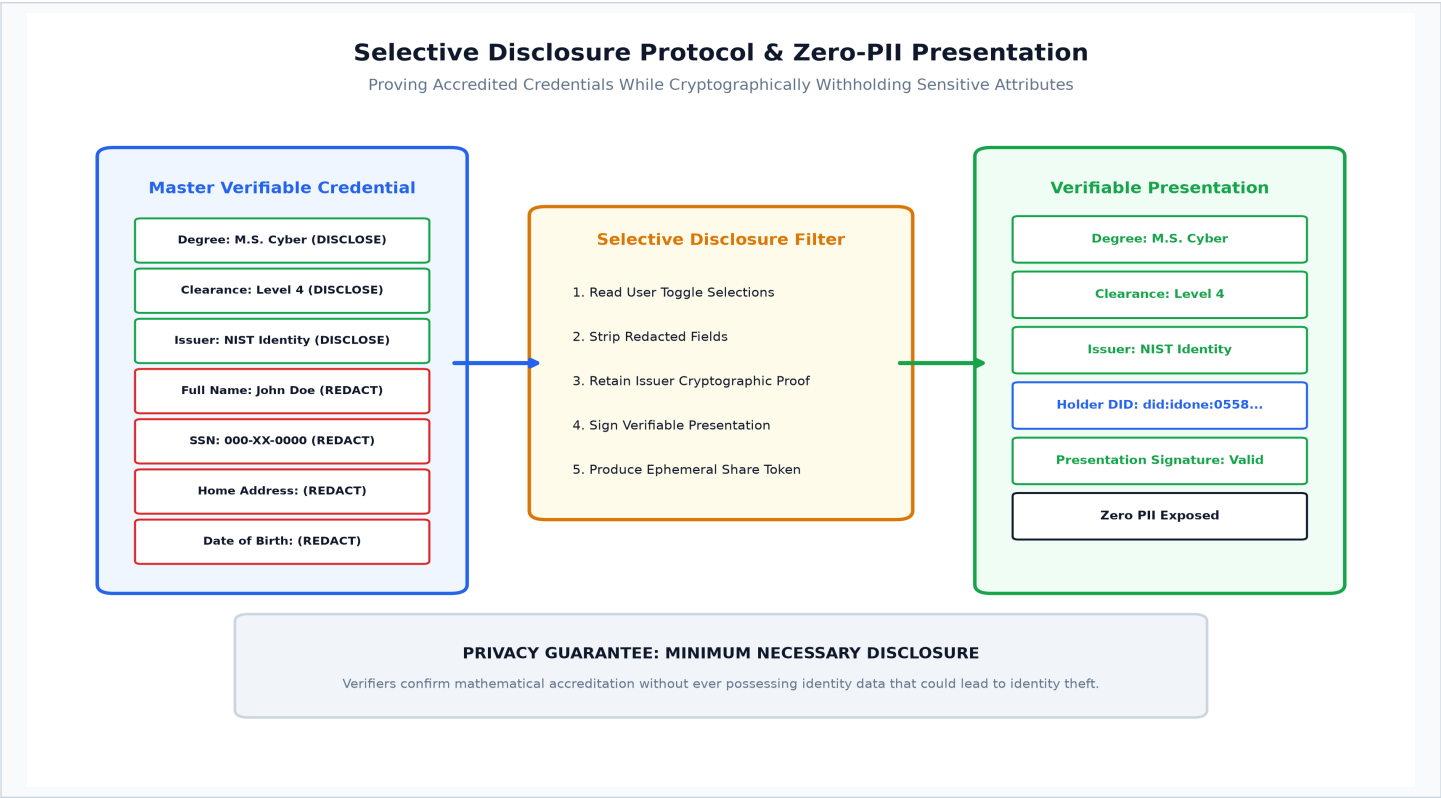
Figure 2.3: RFC 8785 Canonicalization, Ed25519 Signature Generation, and Multi-Step Verification Pipeline

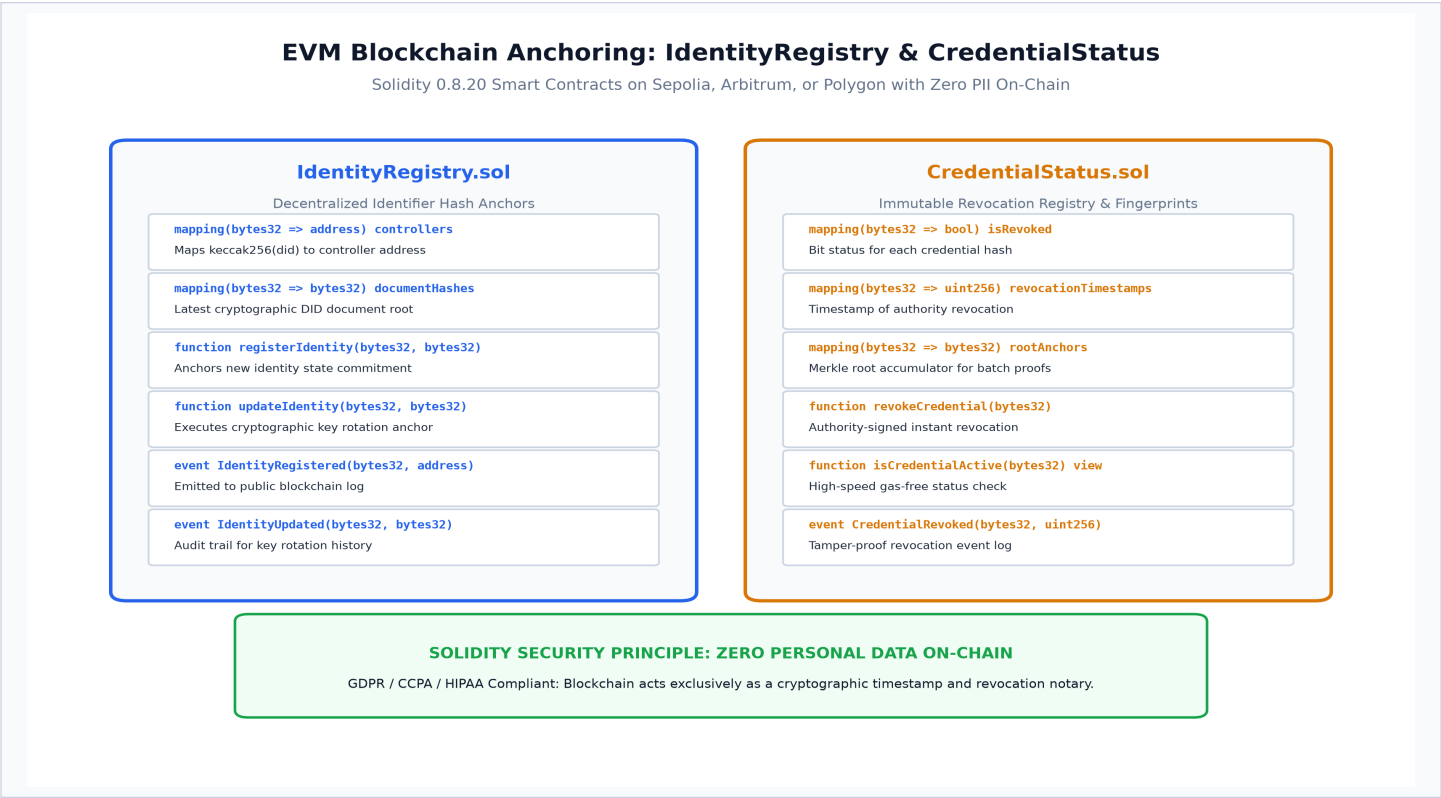
## 4. Zero-Knowledge Client-Side AES-256-GCM Architecture

Many cloud vaults advertise encryption at rest, but store decryption keys on the server or in cloud key management services (KMS). In IDone, all vault encryption occurs strictly in the **client's browser memory** using the native **WebCrypto API**.

**Galois/Counter Mode (GCM) Mathematics:** AES-GCM combines counter-mode confidentiality with polynomial-based Galois message authentication. Each vault item is encrypted using a unique, cryptographically secure 96-bit random Initialization Vector (IV). The operation outputs the ciphertext alongside a 128-bit authentication tag. If even a single bit of ciphertext or IV is tampered with, the client decryption throws a cryptographic exception.







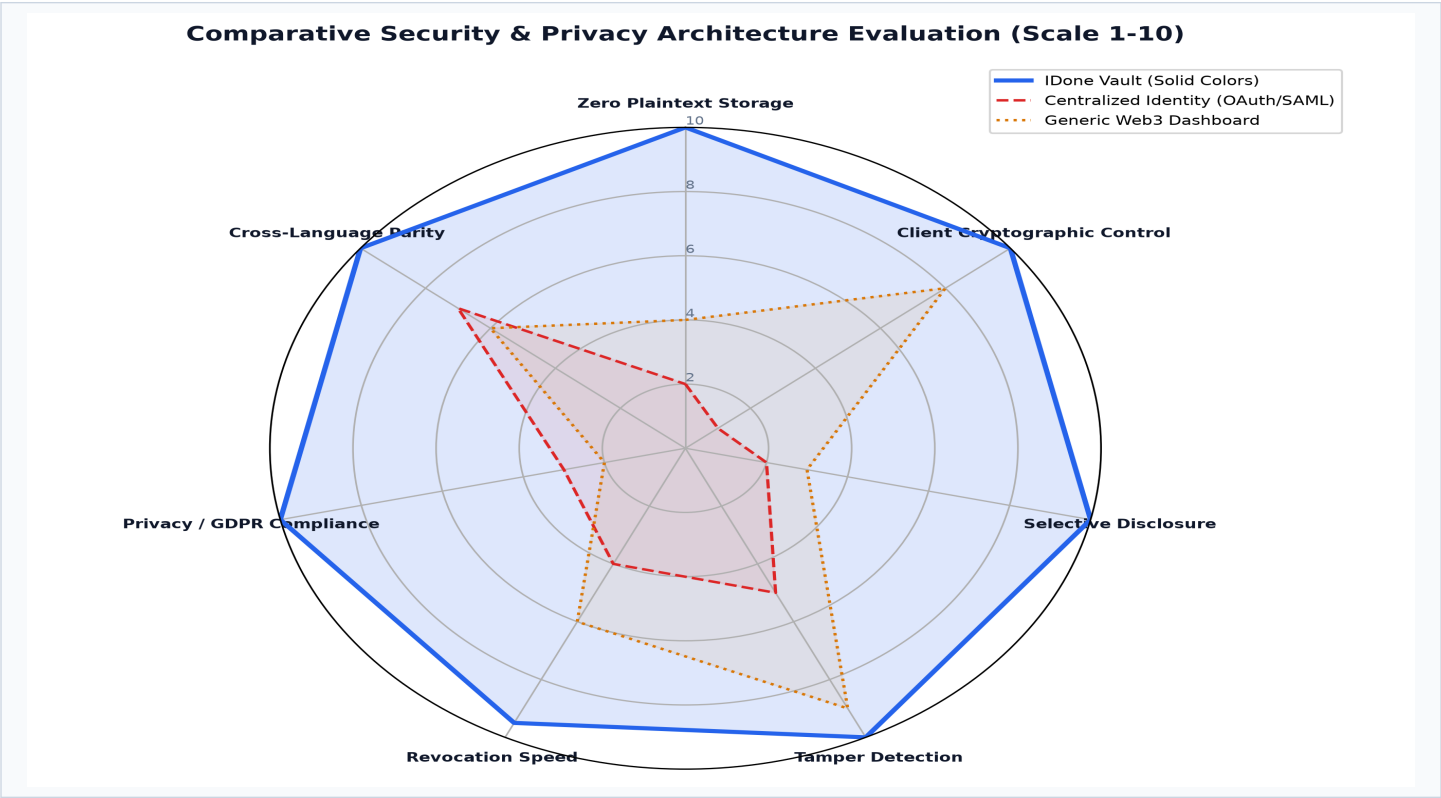


Figure 2.7: Comparative Security Radar Chart: IDone Vault vs Centralized Identity vs Speculative Web3